

Séance 2 — Structures

Introduction à la programmation Python — IUT 1re année

Support de cours

Séance 2

Structures

Pourquoi des fonctions ?

Une **fonction** regroupe des instructions réutilisables sous un nom, pour éviter de dupliquer du code.

```
def convertir_c_en_k(temperature_c): """Convertit une température de °C en K."""  
    resultat = temperature_c + 273.15  
    return resultat  
t = convertir_c_en_k(20) # t = 293.15
```

Paramètres et valeurs par défaut

```
def convertir(valeur, unite="celsius"): if unite == "celsius": return valeur +  
    273.15 elif unite == "fahrenheit": return (valeur - 32) * 5/9 + 273.15  
convertir(20) # utilise unite="celsius" par défaut  
convertir(68, unite="fahrenheit")
```

Fonctions avec / sans valeur de retour

```
def calculer_moyenne(liste): # AVEC retour : renvoie un résultat utilisable  
    return sum(liste) / len(liste)  
def afficher_alerte(valeur, seuil): # SANS retour  
    (procédure) : produit un effet (affichage)  
    if valeur > seuil: print("ALERTE : seuil dépassé")
```

Portée des variables

Une variable créée **dans** une fonction (variable *locale*) n'existe pas en dehors.

```
def f(): x = 1 # locale à f  
    return x  
x = 5  
f() # affiche toujours 1, pas 5
```

Listes

Une **liste** est une séquence **modifiable** et **ordonnée** de valeurs.

```
mesures = [12.3, 14.1, 13.8] mesures.append(15.0) # ajoute un élément mesures[0] = 12.5 # modifie un élément print(len(mesures)) # nombre d'éléments : 4
```

Tuples

Un **tuple** ressemble à une liste mais est **non modifiable** (immuable) — utile pour des données fixes (coordonnées, constantes).

```
point = (48.85, 2.35) # latitude, longitude x, y = point # dépaquetage : x = 48.85, y = 2.35
```

Indexage et découpage (slicing)

Valable sur listes, tuples et chaînes de caractères.

```
liste = [10, 20, 30, 40, 50] liste[0] # 10 (premier élément) liste[-1] # 50 (dernier élément) liste[1:3] # [20, 30] (du index 1 inclus au 3 exclu) liste[:2] # [10, 20] liste[::2] # [10, 30, 50] (un élément sur deux)
```

Fonctions Python utiles sur les séquences

```
len([1, 2, 3]) # 3 sum([1, 2, 3]) # 6 max([1, 5, 2]) # 5 min([1, 5, 2]) # 1 sorted([3, 1, 2]) # [1, 2, 3] "a" in ["a", "b"] # True (test d'appartenance)
```

Manipulation de texte

```
prenom = "Alice" nom = "Dupont" nom_complet = prenom + " " + nom # concaténation initiale = prenom[0] # indexage : "A" debut = nom_complet[0:5] # découpage : "Alice"
```

Caractères spéciaux : `\n` (nouvelle ligne), `\t` (tabulation), `\"` (guillemet échappé)

Formatage de texte (f-strings)

```
valeur = 3.14159 print(f"Pi vaut environ {valeur:.2f}") # "Pi vaut environ 3.14" print(f"{valeur = }") # affiche "valeur = 3.14159"
```

Chercher, remplacer, séparer, joindre

```
phrase = " Bonjour le monde " phrase.strip() # enlève les espaces au début/fin
phrase.replace("monde", "IUT") # remplace un mot phrase.find("monde") # position
de "monde" (ou -1 si absent) "a,b,c".split(",") # ["a", "b", "c"] : découpe en
liste "-".join(["a", "b", "c"]) # "a-b-c" : assemble une liste en texte
```

Boucle **for**

```
for i in range(5): # range(5) génère 0, 1, 2, 3, 4 print(i) for mesure in [12.3,
14.1, 13.8]: # parcourt directement une liste print(mesure)
```

Boucle **while**

```
x = 10 while x > 0: print(x) x -= 1
```

break **et** **continue**

```
for i in range(10): if i == 5: break # arrête complètement la boucle if i % 2 ==
0: continue # passe directement au tour suivant print(i) # affiche 1, 3
```

À vous de jouer

Direction les exercices de la séance 2 →

Exercices

Exercices — Séance 2 : Structures

Ex. 1 — Fonction sans `max()` Écrire une fonction `maximum(liste)` qui renvoie le plus grand élément, sans utiliser `max()`, à l'aide d'une boucle.

Ex. 2 — Fonctions de conversion Écrire une fonction `convertir(valeur, unite="m")` gérant au moins 3 unités de longueur (m, cm, km) avec une valeur par défaut.

Ex. 3 — Tuple et dépaquetage Créer un tuple de coordonnées GPS (`latitude, longitude`), le dépaqueter dans deux variables, et afficher un message formaté.

Ex. 4 — Découpage de texte À partir d'une phrase saisie par l'utilisateur, afficher : les 5 premiers caractères, les 5 derniers, un mot sur deux (via `split()` et slicing de liste).

Ex. 5 — Nettoyage de texte À partir d'une chaîne du type `" Alice ; Bob ;Charlie "`, obtenir la liste `["Alice", "Bob", "Charlie"]` propre (sans espaces) en utilisant `strip`, `split`, et une boucle ou une compréhension simple.

Ex. 6 — Recherche avec `while/break` Écrire un programme qui parcourt une liste de mesures avec `while` et s'arrête (`break`) dès qu'une valeur dépasse un seuil, en affichant sa position (utiliser `continue` pour ignorer les valeurs négatives).